

Access Modifiers in Java

What are Access Modifiers?

Access modifiers are keywords that can be used to control the visibility of fields, methods, and constructors in a class. The four access modifiers in Java are public, protected, default, and private.

You must have seen public, private and protected keywords while practicing java programs, these are called access modifiers. There are two types of modifiers in Java: **access modifiers** and **non-access modifiers**.

As the name suggests access modifiers in Java helps to restrict the scope of a class, constructor, variable, method, or data member. There are four types of access modifiers available in java:

1. Default – No keyword required
2. Private
3. Protected
4. Public

Understanding Java Access Modifiers

Let's understand the access modifiers in Java by a simple table.

Access Modifier	within class	within package	outside package by subclass only	outside package
Private	Y	N	N	N
Default	Y	Y	N	N
Protected	Y	Y	Y	N
Public	Y	Y	Y	Y

Private: The access level of a private modifier is only within the class. It cannot be accessed from outside the class.

Default: The access level of a default modifier is only within the package. It cannot be accessed from outside the package. If you do not specify any access level, it will be the default.

When no access modifier is specified for a class, method, or data member – It is said to be having the default access modifier by default.

The data members, class or methods which are not declared using any access modifiers i.e. having default access modifier are accessible only within the same package.

Protected: The access level of a protected modifier is within the package and outside the package through child class. If you do not make the child class, it cannot be accessed from outside the package.

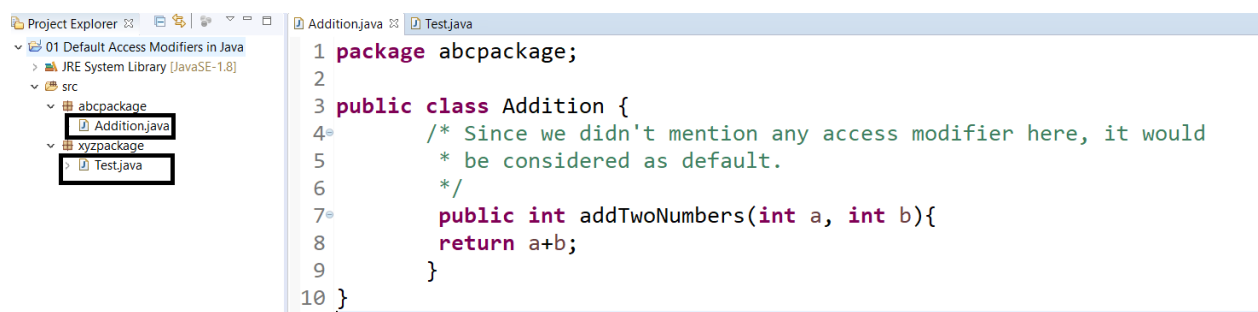
Public: The access level of a public modifier is everywhere. It can be accessed from within the class, outside the class, within the package and outside the package.

There are many **non-access modifiers**, such as static, abstract, synchronized, native, volatile, transient, etc. Here, we are going to learn the access modifiers only.

Default Example 1:

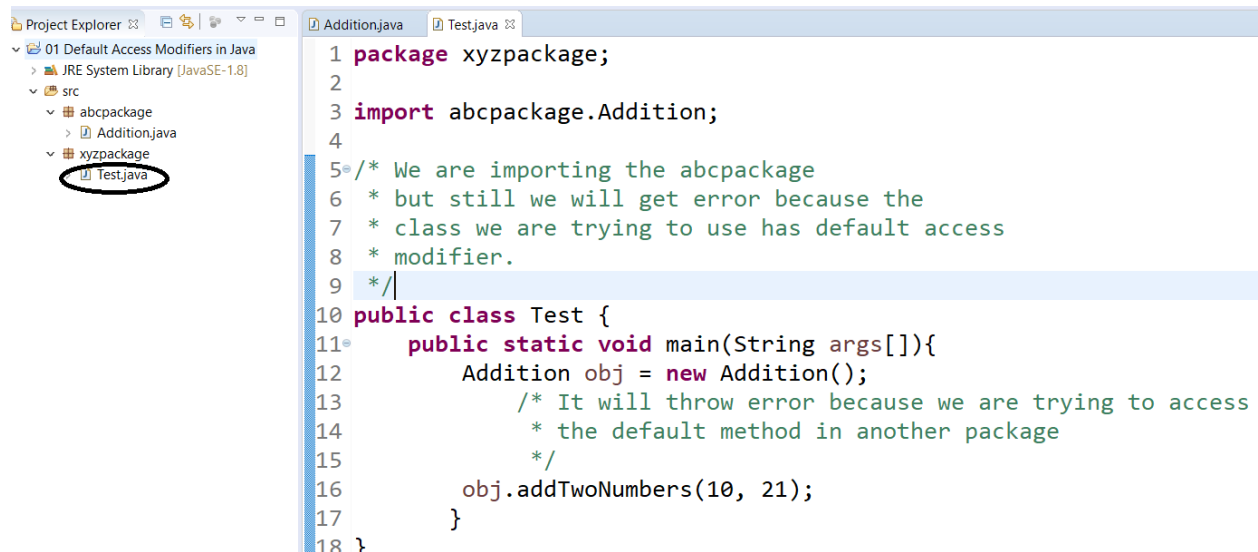
It is not a keyword. Any Java members such as class or methods or data members when not specified with any access modifier they are by default considered as default access modifiers. These methods or data members are only accessible within the same package and they cannot be accessed from outside the package. It provides more visibility than a private access modifier. But this access modifier is more restricted than protected and public access modifiers.

In this example, we will create two packages and the classes in the packages will be having the default access modifiers and we will try to access a class from one package from a class of the second package.



The screenshot shows an IDE with the Project Explorer on the left and the code editor on the right. In the Project Explorer, the 'src' folder is expanded, showing two packages: 'abcpackage' and 'xyzpackage'. Under 'abcpackage', the file 'Addition.java' is highlighted with a red box. The code editor shows the content of 'Addition.java'.

```
1 package abcpackage;
2
3 public class Addition {
4     /* Since we didn't mention any access modifier here, it would
5      * be considered as default.
6      */
7     public int addTwoNumbers(int a, int b){
8         return a+b;
9     }
10 }
```



The screenshot shows the same IDE with the Project Explorer on the left and the code editor on the right. In the Project Explorer, the 'xyzpackage' folder is expanded, and the file 'Test.java' is highlighted with a red box. The code editor shows the content of 'Test.java'.

```
1 package xyzpackage;
2
3 import abcpackage.Addition;
4
5 /* We are importing the abcpackage
6 * but still we will get error because the
7 * class we are trying to use has default access
8 * modifier.
9 */
10 public class Test {
11     public static void main(String args[]){
12         Addition obj = new Addition();
13         /* It will throw error because we are trying to access
14          * the default method in another package
15          */
16         obj.addTwoNumbers(10, 21);
17     }
18 }
```

```
package xyzpackage;
/* We are importing the abcpackage
 * but still we will get error because the
 * class we are trying to use has default access
 * modifier.
 */
```

```
public class Test {
    public static void main(String args[]){
        Addition obj = new Addition();
```

Addition cannot be resolved to a type

11 quick fixes available:

- ⚡ Import 'Addition' (abcpackage)
- 📄 Create class 'Addition'
- 📄 Create interface 'Addition'
- 🔗 Change to 'Action' (java.awt.Desktop)
- 🔗 Change to 'Action' (javax.swing)
- 🔗 Change to 'Action' (javax.xml.ws)
- 🔗 Change to 'Annotation' (java.lang.annotation)
- 🔗 Change to 'Annotation' (java.text)
- 📄 Create enum 'Addition'

because we are trying to access
in another package

Default Example 2:

Let us consider an example for the default access modifier.

However, it's not available in other packages.

```
package com.DefaultAccessModifiers;
```

```
public class SuperPublic {
```

```
    static void defaultMethod()
```

```
    // defaultMethod() is accessible in another class of the same package:
```

```
{
    System.out.println("Hi, I'm Default Access Modifier");
}
```

```
    public class Public {
```

```
        public Public() {
```

```
            SuperPublic.defaultMethod();
```

```
// Available in the same package.
```

```
        }
    }
}
```

2. Private Access Modifier

Any method, property or constructor with the **private keyword** is accessible from the same class only. This is the most restrictive access modifier and is core to the concept of encapsulation. All data will be hidden from the outside world:

The scope of private modifier is limited to the class only.

1. Private Data members and methods are only accessible within the class
2. Class and Interface cannot be declared as private
3. If a class has private constructor then you cannot create the object of that class from outside of the class.

Let's see an example to understand this:

```
package com.PrivateAccessModifier;

public class SuperPublic {
    static private void privateMethod() {
        System.out.println("Hi, I'm Private Access Modifier");
    }

    @SuppressWarnings("unused")
    private void anotherPrivateMethod() {
        privateMethod(); // available in the same class only.
    }
}
```

3. Public Access Modifier

If we add the public keyword to a class, method or property then we're making it available to the whole world, i.e. all other classes in all packages will be able to use it. This is the least restrictive access modifier:

publicMethod() is available in another package:

```
package com.PublicAccessModifiers;

public class SuperPublic {
    public static void publicMethod() {
        System.out.println("Hi, I'm Public Access Modifier");
    }
}
```

Calling from Another Package

```
package com.PublicAccessModifiers.another;

import com.PublicAccessModifiers.SuperPublic;

public class AnotherPublic {
    public AnotherPublic() {
        SuperPublic.publicMethod();
        // Available everywhere. Let's note different package.
    }

    public static void main(String arg[]) {
        System.out.println("Hi, Public Access Modifier from another Package");
    }
}
```

4. Protected Access Modifier

Between public and private access levels, there's the protected access modifier.

If we declare a method, property or constructor with the protected keyword, we can access the member from the same package (as with package-private access level) and in addition from all subclasses of its class, even if they lie in other packages:

```
package com.ProtectedAccessModifiers;
public class SuperPublic {
    static protected void protectedMethod() {
        System.out.println("Hi, I'm Protected Access Modifier");
    }
}

// Available in subclass. Let's note different package.
package com.ProtectedAccessModifiers.Another;
import com.ProtectedAccessModifiers.SuperPublic;

public class AnotherSubClass extends SuperPublic {
    public AnotherSubClass() {
        SuperPublic.protectedMethod();
        // Available in subclass. Let's note different package.
    }

    public static void main(String arg[]) {
        System.out.println("Protected Access Modifier - Available in Subclass");
    }
}
```